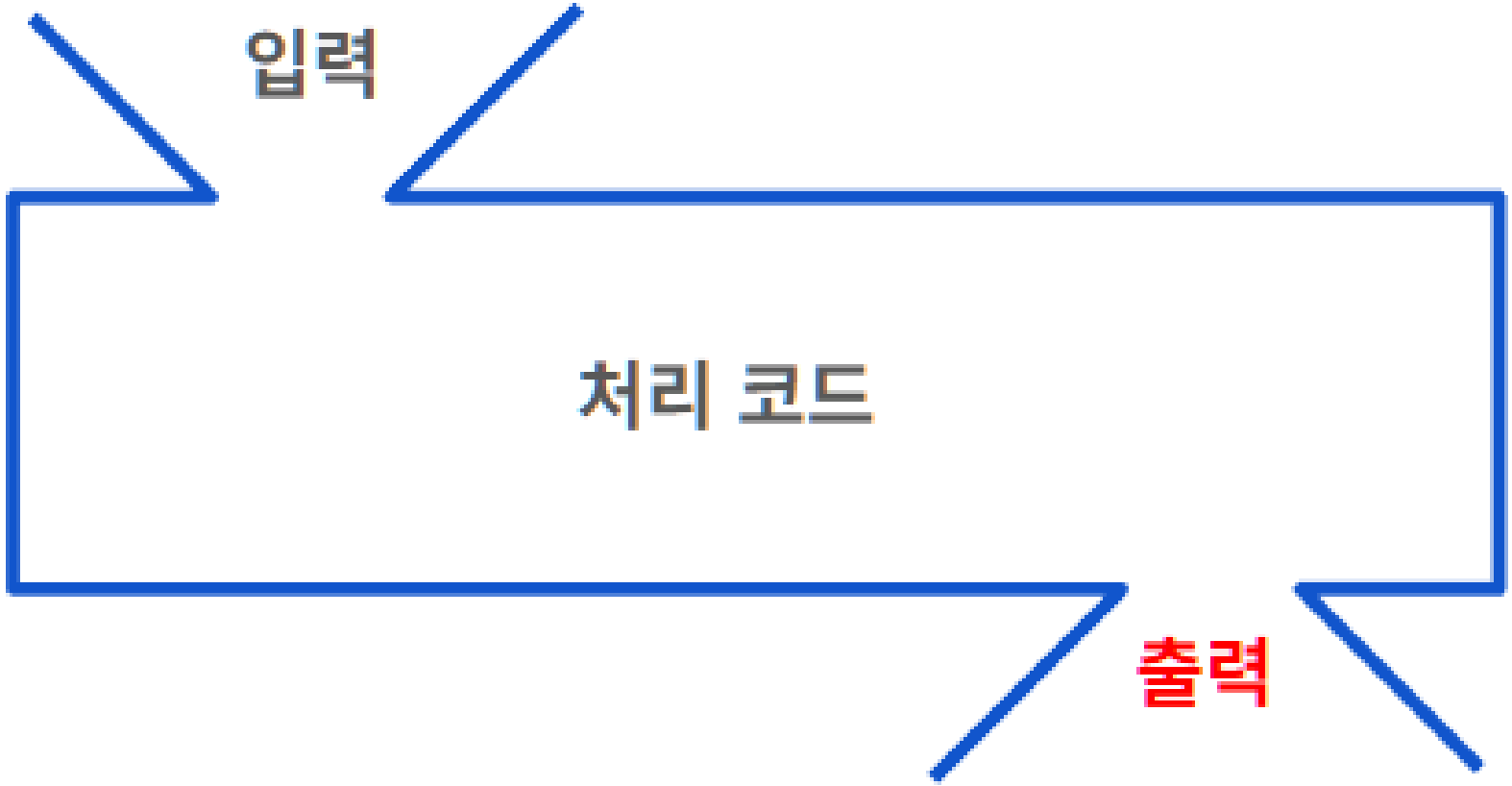
Copyright All rights reserved

05. 함수 (Function)

Heesung Oh
https://github.com/3dapi

Heesung Oh
Copyright All rights reserved

- 프로그램 실행의 기본 단위
- 입력 → 실행 → 결과



5.1 함수 정의(Function Definition)

Heesung Oh
Copyright All rights reserved

- 함수구성 요소
  - 반환형(Return type)
    - 함수가 결과로 돌려주는 값의 자료형
  - 함수 이름(Function name)
    - 식별자, 호출할 때 사용할 이름
  - 매개변수 목록(Parameter list)
    - 함수에 전달되는 입력 값의 자료형과 변수명 목록
  - 함수 본문(Body)
    - 함수의 목적에 맞는 실행 문.
    - 중괄호 { } 안에 실행 명령을 나열
  - return
    - 호출한 곳으로 결과 반환

5.1 함수의 선언과 정의

Heesung Oh
Copyright All rights reserved

- **함수 선언 (Declaration)**

```
반환형 함수이름(매개변수 목록)
{
    실행문;
    return 반환 값;
}
```

- **함수 정의 (Define)**

```
반환형 함수이름(매개변수 목록)
{
    실행문;
    return 반환 값;
}
```

5.1 함수의 선언과 정의 – 함수의 구현

Heesung Oh
Copyright All rights reserved

- 반환형(Return type)
  - 함수가 결과로 돌려주는 값의 자료형 선언과 일치
- 함수 이름(Function name)
  - 식별자, 호출할 때 사용할 이름. 같은 이름 허용 안함
  - ※ C++은 오버로딩이 있어 인수의 개수나 타입이 다르면 중복 허용함.
- 매개변수 목록(Parameter list)
  - 함수에 전달되는 입력 값의 자료형과 변수명 목록
  - 함수 선언에서 작성한 매개변수 자료형, 개수 일치
  - 자료형만 일치하면 컴파일이 가능. 코드의 가독성과 유지보수를 위해 선언과 정의를 일치
- 함수 본문(Body)
  - 함수의 목적에 맞는 실행 문이 작성되는 영역
  - 중괄호 { } 안에 실행 명령 나열
- return 문
  - 함수의 실행을 종료, 결과 값을 호출한 곳으로 반환

5.1 함수의 선언과 정의 – 예

Heesung Oh
Copyright All rights reserved

- 함수 선언 (Declaration)

```
// ShowCharacterStat 함수 선언
void ShowCharacterStat(float playerHp, float playerMp);
```

- 함수 정의 (Define)

```
// ShowCharacterStat 함수 정의
void ShowCharacterStat(float playerHp, float playerMp)
{
    printf("Player Stat  ==============\n");
    printf("HP: %f\n", playerHp);
    printf("MP: %f\n", playerMp);
}
```

Heesung Oh
Copyright All rights reserved

- 함수 호출

```
// 함수 선언: 호출보다 앞에 있어야 한다.
반환형 함수이름(매개변수 목록);

// 어떤 함수에서 호출
어떤_함수()
{
    함수이름(인수 목록);
}

// 함수 정의 (어딘가에서 반드시 정의되어야 한다.)
반환형 함수이름(매개변수 목록)
{
    ...;
}
```

Heesung Oh
Copyright All rights reserved

- 함수 호출

```
// 함수 선언: 호출보다 앞에 있어야 한다.
반환형 함수이름(매개변수 목록);

// 응용_함수에서 호출
응용_함수()
{
    함수이름(인수 목록);
}

// 함수 정의 (프로젝트에 포함된 임의의 소스 파일에서 반드시 정의)
반환형 함수이름(매개변수 목록)
{
    ...;
}
```

Heesung Oh
Copyright All rights reserved

- **함수 호출**

```c
#include <stdio.h>
int Add(int a, int b); // Add 함수 선언
int Multi(int a, int b); // Multi 함수 선언

int main(void)
{
    int x = 5000, y = 4000;
    // Add 함수 호출
    int z = Add(x, y);
    printf("Add(%d, %d) => %d\n", x, y, z);

    // Multi 함수 호출
    // 정의가 없으면 링크 단계에서 오류 발생
    z = Multi(x, y);
    printf("Multi(%d, %d) => %d\n", x, y, z);

    return 0;
}
```

5.3 매개변수/인수

Heesung Oh
Copyright All rights reserved

- 매개변수(Parameter)
  - 함수의 선언이나 정의에서 선언된 변수
  - 함수 호출 시 전달되는 값을 받는 변수

```
int Add(int a, int b); // a, b : 매개변수
```

- 인수 (Argument)
  - 함수를 호출할 때 전달하는 실제 값
  - 매개변수에 전달되는 값

```
int z = Add(x, y); // x, y : 인수
```

- void 매개변수
  - func() → 매개변수 정보 없음
  - func(void) → 매개변수 없음
    → 인수 개수 안 맞으면 Warning 또는 Error

5.3.3 값 전달 방식과 변수의 생명 시간

Heesung Oh
Copyright All rights reserved

- 값 전달 방식 (Call by Value)
  - C 언어 기본 함수 호출 방식
  - 인수(argument)의 값 → 매개변수(parameter)로 복사 전달

- 변수의 생명 시간 (Lifetime)
  - 매개변수(parameter), 지역 변수(local variable)
  - 생성
    - 함수 호출 시 생성
  - 소멸
    - 함수 실행 종료 시 소멸
  - 저장소 위치
    - Stack 영역

5.4 반환 값 (Return Value)

Heesung Oh
Copyright All rights reserved

- 반환 값 (Return Value)
  - 함수 실행 결과를 호출한 위치로 전달
  - return 문 사용

- return 동작
  - 함수 실행 종료
  - 제어 흐름 → 호출 위치

- 반환 방법
  - 변수 반환
    - Ex) return result;
  - 계산식 반환
    - Ex) return a + b;

5.4 반환 값 (Return Value)

Heesung Oh
Copyright All rights reserved

- void 반환형
  - 반환 값 없음
  - 동작만 수행

```
void ShowMessage(void)
{
    printf("메시지를 출력합니다.\n");
}
```

5.5 랜덤 함수 rand()

Heesung Oh
Copyright All rights reserved

- rand()
  - 헤더 파일: #include ⟨stdlib.h⟩
  - 반환 범위: 0 ~ RAND_MAX
  - RAND_MAX
    - 구현마다 다름. 일반적으로 32767 이상

- 난수 종류
  - 완전한 난수(Random) 아님
  - 알고리즘 기반 의사 난수 (Pseudo-Random)

- 주의
  - RAND_MAX 작음
  - 난수 품질 좋지 않음 → 교육용 / 간단한 테스트 코드에 적합
  - 실제 게임 개발
    - 더 좋은 RNG 사용. 예: C++ Mersenne Twister (std::mt19937)

5.5 랜덤 함수 srand()

Heesung Oh
Copyright All rights reserved

- 난수 공식
  - [0, n-1] : rand() % n
  - 일반 범위: min + rand() % (max - min + 1)
  - 예)
    - rand() % 10 // 0~9
    - 1 + rand() % 6 // 주사위

- srand()
  - 난수 초기값 (Seed)
  - Seed: 난수 시작 값. Seed가 동일하면 → 난수 순서 동일
  - time을 사용한 실행마다 다른 Seed 생성
    - srand((unsigned int)time(NULL));
  - 주의
    - 프로그램 시작 시 한 번만 호출

5.6 타임 함수 time()

Heesung Oh
Copyright All rights reserved

- time()
  - 헤더 파일: #include ⟨time.h⟩
  - 기준 시간: 1970-01-01 00:00:00 (UTC) 이후 경과 시간 → UNIX Time / Epoch Time
  - time(NULL): 현재 시간을 초 단위로 반환

- 난수 seed 설정
  - srand((unsigned int)time(NULL)); → 프로그램 실행마다 다른 난수 시퀀스

- 범위 난수 생성
  - 기본 공식: min + rand() % (max - min + 1)
  - 예: 1 + rand() % 6 → 주사위

- 난수 함수 만들기(minVal, maxVal)

```
int GetRandom(int minVal, int maxVal)
{
    return minVal + rand() % (maxVal - minVal + 1);
}
```

5.7 변수의 범위와 생명 시간

Heesung Oh
Copyright All rights reserved

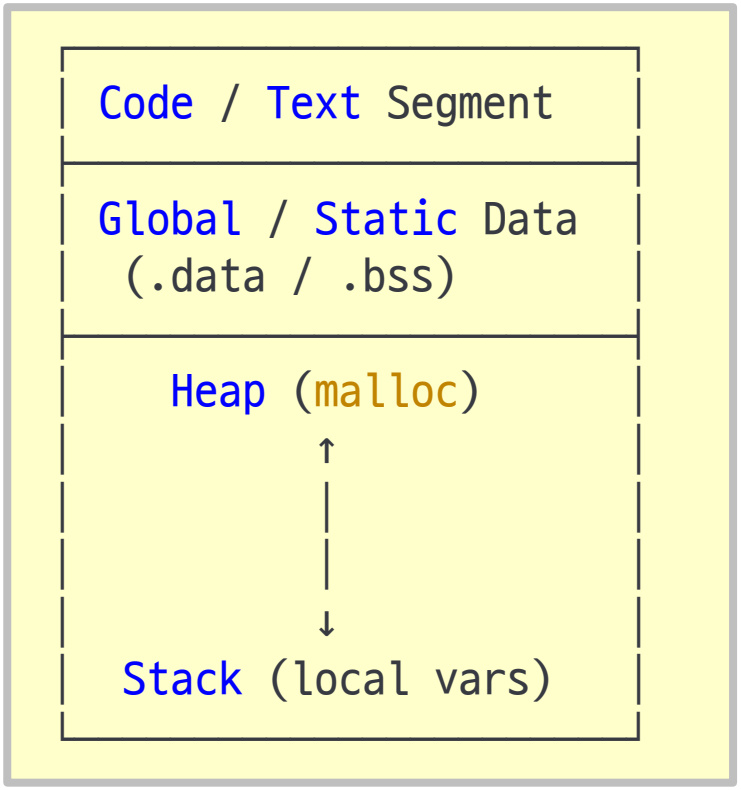
- 변수의 저장 위치, 범위, 생명시간 결정 요소
  - 선언 위치
  - Storage class 키워드

- 대표적인 변수 종류
  - 지역 변수 (Local Variable)
  - 전역 변수 (Global Variable)
  - static 변수
  - register 변수

5.7 프로그램 메모리 구조

Heesung Oh
Copyright All rights reserved

- Code 영역 → 프로그램 코드
- Global / Static → 전역 변수, static 변수
- Heap → 동적 메모리
- Stack → 지역 변수

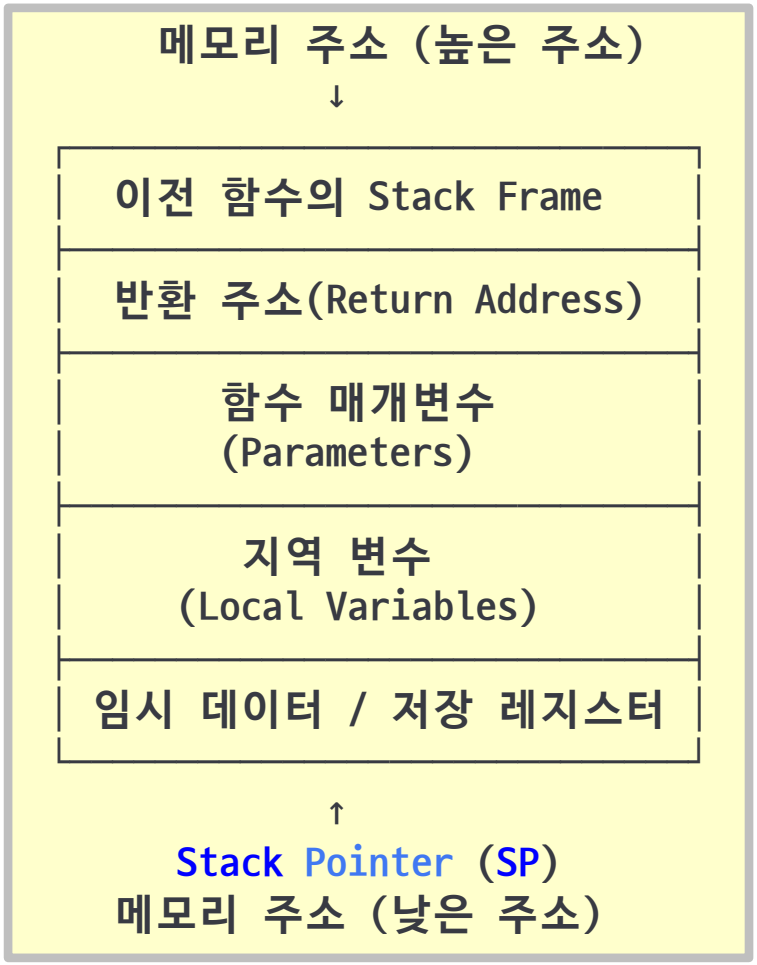


5.7.1 지역변수

Heesung Oh
Copyright All rights reserved

- 지역 변수 (Local Variable)
  - 함수 내부에서 선언
  - Stack 영역에 저장
  - 함수 종료 시 자동 소멸

- Stack 특징
  - 매우 빠른 메모리 접근
    - Stack Pointer로 관리
    - 함수 호출 → 공간 확보
    - 함수 종료 → 자동 해제
  - 높은 캐시 효율
    - 연속된 메모리 구조
    - CPU Cache Hit 높음
  - 크기 제한
    - 운영체제마다 기본 스택 크기 존재
    - 큰 배열 / 깊은 재귀 → Stack Overflow 발생



5.7.2 지역변수 - Scope / Lifetime

Heesung Oh
Copyright All rights reserved

- Scope
  - 변수에 접근 가능한 범위
  - 선언된 블록 내부에서만 사용 가능
  - Shadowing: 내부 블록 변수가 외부 변수를 가림

```
int value = 10;
{
    int value = 20;
}
```

- Lifetime
  - 블록 진입 → 생성
  - 블록 종료 → 소멸

5.7.4 전역 변수(Global Variable)

Heesung Oh
Copyright All rights reserved

- 전역 변수 (Global Variable)
  - 모든 함수 외부에서 선언
  - 프로그램 시작 시 생성
  - 프로그램 종료 시 소멸
- 초기화
  - 기본값으로 자동 초기화
- 주의
  - 여러 곳에서 변경 가능
  - 멀티 스레드 환경에서 위험

5.7.5 static 변수(Global Variable)

Heesung Oh
Copyright All rights reserved

- static 변수
  - 정적 영역에 저장
  - 프로그램 종료까지 유지
- 지역 static
  - Scope → 함수 내부
  - Lifetime → 프로그램 전체
- 전역 static
  - 파일 내부에서만 접근 가능
  - internal linkage

```
프로그램 시작
    | static 변수 생성
    ↓
┌──────────────────────────┐
│ static 변수 (정적 영역)  │
│                          │
│ 프로그램 전체 동안 유지  │
│                          │
│      함수 호출 1         │
│      함수 호출 2         │
│      함수 호출 3         │
│        ...               │
└──────────────────────────┘
    ↑
프로그램 종료 시 소멸5
```

5.7.6 register 변수

Heesung Oh
Copyright All rights reserved

- register 변수
  - CPU 레지스터 사용 힌트
  - 주소 연산자(&) 사용 불가
  - 실제 레지스터 저장 여부는 컴파일러 결정

- 현대 컴파일러
  - 자동 레지스터 할당 수행
  - register 키워드 사용 거의 없음

5.8 재귀 함수

Heesung Oh
Copyright All rights reserved

- 재귀 함수 (Recursion)
  - 자기 자신 호출 방식 함수
- 필수 요소
  - 종료 조건 (Base Case)
  - 재귀 종료 조건
- 재귀 단계 (Recursive Step)
  - 더 작은 문제 → 자기 자신 호출
- 재귀 호출과 스택
  - 재귀 호출 → Stack 메모리 사용
  - 재귀 호출 중첩 횟수 → 재귀 깊이 (Recursion Depth)
  - 재귀 깊이 증가 → Stack 사용량 증가
  - 종료 조건 오류 / 깊이 과다 → Stack Overflow
- 대표적인 재귀 사용 예
  - 파일 시스템 탐색 (Directory Traversal)
  - 트리 구조 탐색 (Tree Traversal)

Heesung Oh
Copyright All rights reserved

- **재귀 함수 형태**

```
반환형 함수이름(매개변수)
{
    if (종료 조건)
        return 어떤 값;

    return 함수이름(더 작은 값);
}
```

- **Sum 알고리즘**

```
Sum(n) = n + Sum(n - 1)
Sum(1) = 1    (종료 조건)
```

5.8 재귀 함수 – 재귀 vs 반복

Heesung Oh
Copyright All rights reserved

- 재귀
  - 코드 구조 간결
  - 알고리즘 표현 직관적
  - 함수 호출 비용
  - Stack 사용 증가

- 반복
  - 함수 호출 없음
  - Stack 사용 없음
  - 성능 안정

5. 학습정리

Heesung Oh
Copyright All rights reserved

- 함수
  - 프로그램 기본 단위, 입력 매개변수, 출력 반환값
  - 함수 구성: 반환형, 함수 이름, 매개변수 목록, 함수 본문, return 문
- 함수 선언(Function Declaration)
  - 함수 형태 사전 알림, 파일 상단 작성
- 매개변수(parameter)
  - 함수 정의 변수
- 인수(argument)
  - 함수 호출 전달 값
- C 언어 전달 방식
  - 값 전달(Call by Value)

5. 학습정리

Heesung Oh
Copyright All rights reserved

- 난수 생성
  - rand()
  - 난수 초기값 srand()
  - seed 값 time()

- 변수 종류
  - 지역 변수 (Local)
  - 전역 변수 (Global)
  - static 변수
  - register 변수

- 변수
  - 범위(Scope)
  - 생명 시간(Lifetime)

5. 실습 및 과제

Heesung Oh
Copyright All rights reserved

- 과제 1
  - 덧셈, 뺄셈, 곱셈, 나눗셈 함수
  - random 함수 이용
- 과제 2
  - 플레이어 체력(HP), 마력(MP)을 전역 변수로 선언, 출력
- 과제 3
  - 몹1, 몹2, 몹3 용 함수를 만들고, 각 몹의 능력치(stat)